Laporan Tugas Besar II: CNN dan RNN/LSTM

Shanice Feodora Tjahjono **Muhammad Fathur Rizky** **Ahmad Wicaksono**
Kelompok `Seq2SeqIsAllYouNeed`, Program Studi Teknik Informatika, Institut Teknologi Bandung
`{13523097,13523105,13523121}@std.stei.itb.ac.id`

Abstrak

Laporan ini membahas scratch implementation untuk *forward propagation* Convolutional Neural Network (CNN), Simple Recurrent Neural Network (Simple RNN), dan Long Short-Term Memory (LSTM) menggunakan NumPy. Bagian CNN dikerjakan pada tugas klasifikasi Intel Image Classification, sedangkan bagian RNN dan LSTM dikerjakan pada tugas *image captioning* Flickr8k dengan arsitektur *pre-inject*. Bobot dilatih menggunakan Keras, kemudian dipindahkan ke scratch implementation untuk menjalankan inferensi dan perbandingan. Implementasi mencakup `Conv2D`, `LocallyConnected2D`, pooling, `Flatten`, `Dense`, `Embedding`, `SimpleRNN`, `LSTM`, fungsi aktivasi, metrik klasifikasi, BLEU-4, METEOR, serta pemuat bobot Keras. Pada konfigurasi eksperimen ringan lima epoch, CNN terbaik memperoleh macro F1 sebesar 0.7471 pada variasi tiga layer konvolusi dengan filter 8-16-24, kernel 5, dan max pooling. Scratch implementation CNN menghasilkan macro F1 yang sama dengan Keras pada arsitektur evaluasi `Conv2D`, yaitu 0.6841, dengan kesepakatan prediksi 1.0. Variasi `LocallyConnected2D` dengan arsitektur sepadan memperoleh macro F1 0.5943. Pada *image captioning*, LSTM satu layer dengan hidden state 64 menghasilkan *validation loss* terendah, yaitu 3.6527, sedangkan evaluasi scratch implementation LSTM menunjukkan METEOR sekitar 0.1654 pada *greedy decoding*. Hasil ini menunjukkan bahwa pemindahan bobot dari Keras ke scratch implementation NumPy dapat dilakukan secara konsisten walaupun kualitas caption masih terbatas karena konfigurasi eksperimen dibuat kecil agar waktu komputasi tetap realistis.

1 Deskripsi Persoalan

Tugas Besar II IF3270 Pembelajaran Mesin berfokus pada pemahaman mekanisme *forward propagation* pada CNN, Simple RNN, dan LSTM. Dalam tugas ini, model tidak hanya dipakai sebagai kotak hitam, tetapi juga dibongkar sampai level operasi numerik yang membentuk layer-layernya. Karena itu, scratch implementation menjadi bagian utama, sedangkan Keras dipakai untuk pelatihan bobot dan pembanding hasil inferensi.

Bagian pertama adalah klasifikasi gambar menggunakan dataset Intel Image Classification. Dataset ini berisi gambar alam dan perkotaan dengan enam kelas, yaitu *buildings*, *forest*, *glacier*, *mountain*, *sea*, dan *street*. CNN cocok untuk klasifikasi citra karena memanfaatkan lokalitas spasial, parameter sharing, dan hierarki fitur dari tepi sederhana sampai pola visual yang lebih kompleks [13, 5]. Model CNN dilatih dengan Keras, kemudian bobotnya dimuat ke implementasi NumPy. Persoalan ini dipakai untuk menguji layer konvolusi berbagi parameter, konvolusi tanpa berbagi parameter, pooling, global pooling, *flatten*, dense layer, aktivasi, dan perhitungan macro F1.

Bagian kedua adalah *image captioning* menggunakan dataset Flickr8k. Setiap gambar memiliki beberapa caption rujukan. Persoalan ini menggabungkan representasi visual dari CNN dengan pemodelan urutan bahasa menggunakan RNN atau LSTM [20]. Fitur gambar diekstraksi lebih dahulu menggunakan CNN pralatih yang dibekukan, kemudian diproyeksikan sebagai masukan pertama decoder. Decoder yang diuji adalah Simple RNN dan LSTM. Arsitektur mengikuti skema *pre-inject*: fitur CNN dimasukkan sebagai timestep awal sebelum token `<start>`. Dengan demikian, tugas captioning menguji layer `Embedding`, `SimpleRNN`, `LSTM`, dense projection, dense output, *teacher forcing*, *greedy decoding*, dan *beam search*.

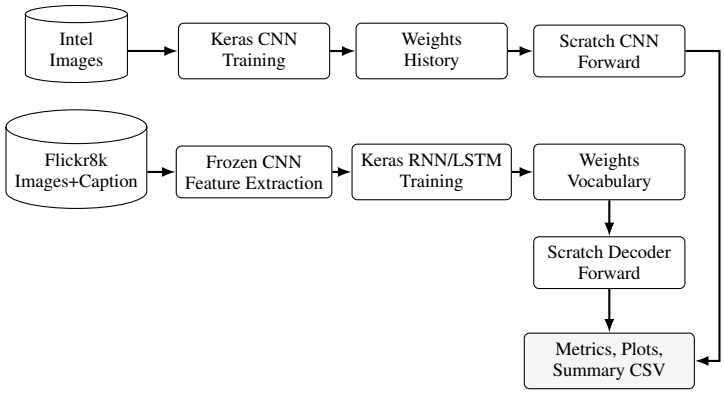
2 Pembahasan

Proyek ini disusun sebagai kerangka kerja kecil yang menyerupai API Keras. Direktori `src/seq2seq` berisi implementasi inti, sedangkan `src/experiments` dan `scripts` berisi pipeline eksperimen. Pemisahan ini dibuat agar layer scratch tetap dapat diuji secara terpisah dari kode pelatihan Keras. Pada sisi CNN, Keras dipakai untuk melatih arsitektur pembanding, sedangkan kelas-kelas NumPy dipakai untuk menjalankan *forward pass* dari bobot yang sama. Pada sisi captioning, Keras dipakai untuk melatih decoder RNN/LSTM, kemudian bobot embedding, recurrent cell, dan dense layer dipindahkan ke scratch implementation.

Keputusan memakai Keras untuk pelatihan dan NumPy untuk inferensi pada scratch implementation sesuai dengan batasan tugas. Implementasi *forward propagation* tidak bergantung pada Keras, sedangkan komponen di luar implementasi inti, seperti pelatihan bobot, ekstraksi fitur CNN pralatih, dan penyimpanan artifact, tetap dapat memanfaatkan library yang tersedia. Pendekatan ini juga membuat validasi lebih jelas karena keluaran scratch implementation NumPy dapat dibandingkan langsung dengan keluaran Keras untuk bobot yang sama.

Secara teoretis, bagian CNN menekankan dua bias induktif utama: konektivitas lokal dan parameter sharing. Konektivitas lokal membatasi operasi pada patch spasial kecil, sedangkan parameter sharing membuat kernel yang sama diterapkan di banyak lokasi gambar [13, 5]. Pada bagian sekuens, Simple RNN menyimpan konteks dalam hidden state. Sementara itu, LSTM menambahkan cell state dan gate untuk membuat jalur memori yang lebih stabil [4, 6].

Gambar 1 memperlihatkan alur umum proyek. Bagian yang dilatih memakai Keras menghasilkan bobot dan riwayat pelatihan. Bobot tersebut kemudian dimuat ke layer NumPy untuk menjalankan inferensi pada scratch implementation. Dengan pemisahan ini, eksperimen dapat tetap realistis secara komputasi, tetapi bagian yang diwajibkan dalam tugas, yaitu *forward propagation* CNN, RNN, dan LSTM pada scratch implementation, tetap dapat diuji secara eksplisit.



Gambar 1: Alur umum eksperimen. Keras dipakai untuk melatih bobot, sedangkan scratch implementation NumPy dipakai untuk validasi *forward propagation*.

3 Penjelasan Implementasi

3.1 Struktur Kode

Struktur kode utama dapat diringkas sebagai berikut.

```
src/
  seq2seq/
    layers/
      convolutional/      # Conv2D dan LocallyConnected2D
      pooling/            # Max, average, global max, global average
      reshaping/          # Flatten
      core/               # Dense dan Embedding
      rnn/                # SimpleRNN dan LSTM
    ops/                  # Kernel NumPy untuk conv, pooling, dan rnn
    saving/               # Loader bobot Keras
    metrics/              # Macro F1, BLEU-4, dan METEOR
    utils/                # Image loader, tokenizer, padding, dataset utils
  experiments/
    cnn/                  # Pembanding Keras CNN
    captioning/           # Decoder captioning dan loader bobot
scripts/
  train_cnn.py
  eval_cnn_scratch.py
  extract_features.py
  build_vocab.py
  train_caption.py
  eval_caption.py
  run_experiment_matrix.py
  summarize_results.py
```

Kelas dasar `Layer` dan `Model` menyediakan antarmuka pemanggilan yang seragam. `Sequential` menyusun beberapa layer secara berurutan, sedangkan model captioning ditulis sebagai turunan `Model` karena memiliki dua input, yaitu fitur gambar dan token caption. Modul `saving` menjadi jembatan antara Keras dan scratch implementation. Modul ini berisi fungsi seperti `load_conv2d`, `load_locally_connected2d`, `load_dense`, `load_embedding`, `load_simple_rnn_cell`, dan `load_lstm_cell`.

3.2 Desain Antarmuka Layer dan Model

Setiap layer dibuat sebagai objek yang dapat dipanggil langsung. Pola ini meniru cara kerja Keras, tetapi isi komputasinya tetap ditulis dengan NumPy. Pada Keras, layer merupakan objek callable yang memiliki logika komputasi melalui `call()`, state berupa bobot, serta antarmuka seperti `get_weights()` dan `set_weights()` untuk mengambil atau mengatur bobot sebagai array

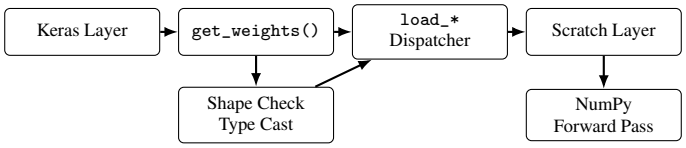
[11]. Sebuah layer menyimpan parameter seperti kernel, bias, ukuran kernel, stride, padding, dan fungsi aktivasi. Saat dipanggil, layer menerima array NumPy dan mengembalikan array NumPy baru. Dengan pola ini, pengujian tiap layer dapat dilakukan secara independen tanpa harus membangun model lengkap.

```
class Layer:
    def __call__(self, inputs):
        return self.call(inputs)

    def call(self, inputs):
        raise NotImplementedError

class Sequential(Model):
    def call(self, inputs):
        output = inputs
        for layer in self.layers:
            output = layer(output)
        return output
```

Kelas `Sequential` dipakai untuk model yang benar-benar linear dari satu input ke satu output, misalnya CNN klasifikasi. Untuk captioning, kelas `CaptioningDecoder` dibuat sebagai model khusus karena membutuhkan dua input: fitur gambar dan urutan token. Desain ini membuat kode tetap sederhana tanpa memaksakan semua arsitektur masuk ke bentuk `Sequential`.

3.3 Pemindahan Bobot Keras ke Implementasi Scratch

Pelatihan dilakukan di Keras karena tugas ini berfokus pada *forward propagation* di scratch implementation, bukan implementasi lengkap *backpropagation* CNN dan RNN. Setelah model Keras dilatih, bobot disimpan dan dipetakan ke struktur parameter pada layer NumPy. Pemetaan ini harus memperhatikan format internal Keras. Misalnya, bobot `Conv2D` Keras berbentuk $(K_h, K_w, C_{in}, C_{out})$, sedangkan `LocallyConnected2D` berbentuk $(output_rows \cdot output_cols, K_h K_w C_{in}, C_{out})$ [7]. RNN dan LSTM juga memakai format gabungan untuk kernel masukan, kernel rekuren, dan bias [12, 10].



Gambar 2: Alur pemindahan bobot dari layer Keras ke layer scratch implementation.

Setelah bobot dimuat, model *scratch* diberi input yang sama dengan model Keras. Jika implementasi benar, distribusi probabilitas atau label prediksi yang dihasilkan seharusnya sama atau sangat dekat. Untuk CNN, kesetaraan ini diukur dengan kesepakatan prediksi. Untuk captioning, kesetaraan diperiksa melalui hasil decoding dan skor evaluasi.

Cuplikan pemuatan bobot CNN

```
for scratch_layer in scratch.layers:
    if isinstance(scratch_layer, Conv2D):
        weights = keras_model.get_layer(scratch_layer.name).get_weights()
        load_conv2d(scratch_layer, weights)
    elif isinstance(scratch_layer, LocallyConnected2D):
        weights = keras_model.get_layer(scratch_layer.name).get_weights()
        load_locally_connected2d(scratch_layer, weights)
    elif isinstance(scratch_layer, Dense):
        weights = keras_model.get_layer(scratch_layer.name).get_weights()
        load_dense(scratch_layer, weights)
```

Nama layer dibuat sama antara model Keras dan model *scratch*. Dengan begitu, proses pemindahan bobot dapat dilakukan berdasarkan nama layer. Pendekatan ini mengurangi risiko salah urutan ketika arsitektur memiliki beberapa layer sejenis.

3.4 Loop Pelatihan dan Penyimpanan Artifact

Pelatihan CNN dan captioning dilakukan melalui script terpisah agar eksperimen dapat dijalankan ulang dari terminal maupun notebook. Setiap script menyimpan dua jenis artifact utama: bobot model dan riwayat eksperimen. Bobot disimpan sebagai `.weights.h5`, sedangkan riwayat disimpan sebagai JSON. JSON berisi konfigurasi eksperimen, nilai loss per epoch, metrik akhir, dan informasi tambahan seperti nama kelas atau ukuran vocabulary.

Cuplikan pelatihan CNN Keras

```
model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=args.learning_rate),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"],
)
history = model.fit(
    train_x,
    train_y,
    validation_data=(val_x, val_y),
    batch_size=args.batch_size,
    epochs=args.epochs,
    verbose=2,
)
```

Setelah pelatihan, model dievaluasi pada test split. Prediksi Keras diubah menjadi label kelas dengan `argmax`, kemudian macro F1 dan confusion matrix dihitung menggunakan implementasi metrik di proyek.

Cuplikan penyimpanan riwayat eksperimen

```
payload = {
    "config": vars(args),
    "history": {
        k: [float(v) for v in values]
        for k, values in history.history.items()
    },
    "test_macro_f1": test_f1,
    "test_confusion_matrix": matrix.tolist(),
    "class_names": list(splits["train"].class_names),
}
json.dump(payload, handle, indent=2)
```

Format JSON ini membuat hasil eksperimen mudah diringkas tanpa menjalankan ulang pelatihan. Script `summarize_results.py` membaca semua file JSON di `artifacts/`, menggabungkannya menjadi `summary.csv`, dan membuat plot training/validation loss.

3.5 Utility Pemrosesan Gambar

Pemrosesan gambar dilakukan dengan Pillow dan NumPy. Fungsi `load_image` membuka gambar dari path, mengubahnya menjadi RGB, melakukan resize, mengubahnya menjadi array, dan menormalisasi nilai piksel ke rentang $[0, 1]$. Fungsi `load_image_batch` melakukan proses yang sama untuk sekumpulan path dan mengembalikan tensor berukuran (N, H, W, C).

Cuplikan pemuatan gambar

```
def load_image(path, target_size=(150, 150), normalize=True):
    image = Image.open(path).convert("RGB")
    image = image.resize(target_size)
```

```
    array = np.asarray(image, dtype=np.float32)
    if normalize:
        array = array / 255.0
    return array

def load_image_batch(paths, target_size=(150, 150), normalize=True):
    images = [
        load_image(path, target_size=target_size, normalize=normalize)
        for path in paths
    ]
    return np.stack(images, axis=0)
```

Cuplikan tersebut menunjukkan bahwa preprocessing CNN tidak memakai Keras preprocessing. Gambar dibaca satu per satu menggunakan Pillow, kemudian dikonversi menjadi array NumPy. Fungsi batch hanya melakukan `stack` terhadap hasil pemrosesan tiap gambar sehingga bentuk akhirnya konsisten dengan input Keras dan implementasi *scratch*, yaitu channel-last (N, H, W, C).

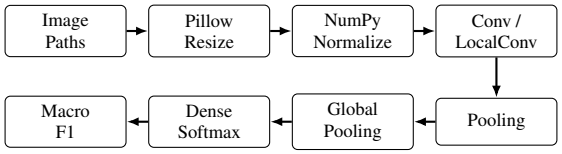
Untuk bagian captioning, fitur gambar diekstraksi dengan CNN pralatih dari Keras. Pada eksperimen ringan, encoder yang digunakan adalah MobileNetV2 agar ekstraksi fitur lebih cepat. MobileNetV2 memakai blok inverted residual dan linear bottleneck untuk memperoleh ekstraktor fitur yang ringan [16]. Script `extract_features.py` juga mendukung `-limit` sehingga eksperimen dapat dijalankan pada subset gambar. Hasil fitur disimpan sebagai `.npy`, sedangkan daftar nama gambar disimpan pada `flickr8k_paths.txt`. Daftar ini kemudian dipakai oleh `build_vocab.py` agar vocabulary dan caption JSON hanya berisi gambar yang benar-benar memiliki fitur.

Cuplikan ekstraksi fitur

```
encoder, preprocess, target_size = _get_encoder(args.encoder)
features = extract_features_with_keras(
    paths,
    encoder,
    target_size=target_size,
    batch_size=args.batch_size,
    preprocess=preprocess,
    save_path=output_path,
)
with Path(args.paths_output).open("w", encoding="utf-8") as handle:
    for path in paths:
        handle.write(f"{path.relative_to(images_dir).as_posix()}\n")
```

Bagian ini memisahkan fitur dan nama gambar. File `.npy` menyimpan matriks fitur, sedangkan `paths-output` menyimpan urutan nama gambar. Pemisahan ini penting karena saat fitur dimuat kembali, setiap baris fitur harus dipasangkan dengan nama gambar yang benar.

Untuk klasifikasi Intel Image Classification, struktur folder dataset dicari secara otomatis dengan beberapa kandidat layout, seperti `seg_train/seg_train` dan `seg_test/seg_test`. Tiap kelas dipetakan ke indeks integer. Data train kemudian diacak untuk membuat validation split sesuai `validation_fraction`. Dengan demikian, pipeline tetap dapat berjalan baik ketika dataset berasal dari KaggleHub ataupun dari path lokal yang sudah disiapkan.



Gambar 3: Alur pemrosesan gambar dan inferensi CNN.

3.6 Forward Propagation CNN

Layer `Conv2D` mengimplementasikan konvolusi dua dimensi dengan parameter yang dibagi untuk seluruh posisi spasial. Parameter sharing membuat jumlah bobot lebih efisien dan memberi bias

induktif terhadap translasi karena pola visual yang sama dapat dideteksi di berbagai lokasi gambar [5, 13]. Untuk input $\mathbf{X}$, kernel $\mathbf{W}$, dan bias $\mathbf{b}$, keluaran pada posisi (i, j) dan kanal output k ditulis sebagai:

$$Y_{i,j,k} = b_k + \sum_{u=0}^{K_h-1} \sum_{v=0}^{K_w-1} \sum_{c=0}^{C-1} X_{i+u,j+v,c} W_{u,v,c,k}. \tag{1}$$

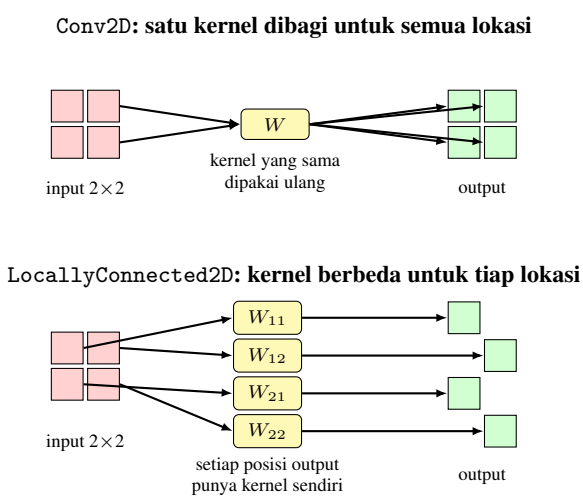
Persamaan tersebut menunjukkan bahwa patch lokal input dikalikan dengan kernel yang sama untuk semua posisi output. Padding `same` dan `valid` ditangani pada kernel NumPy. Setelah operasi konvolusi, fungsi aktivasi seperti ReLU diterapkan bila didefinisikan.

Cuplikan ide utama `Conv2D`

```
for i in range(out_h):
    for j in range(out_w):
        patch = padded[:, i*sh:i*sh+kh, j*sw:j*sw+kw, :]
        for k in range(filters):
            output[:, i, j, k] = np.sum(
                patch * kernel[:, :, :, k],
                axis=(1, 2, 3),
            )
if bias is not None:
    output = output + bias.reshape((1, 1, 1, filters))
```

Potongan ini memperlihatkan konsep konvolusi secara eksplisit. Untuk setiap posisi output, implementasi mengambil *patch* dari input yang sudah dipadding. Patch tersebut dikalikan dengan kernel untuk setiap filter, kemudian dijumlahkan sepanjang dimensi tinggi, lebar, dan kanal. Karena kernel yang sama dipakai pada semua posisi (i, j), layer ini menerapkan *parameter sharing*.

Layer `LocallyConnected2D` memiliki bentuk komputasi yang mirip, tetapi bobotnya tidak dibagi antarlokasi. Bobot layer ini mengikuti format Keras, yaitu $(output_rows \times output_cols, K_h K_w C_{in}, C_{out})$. Karena setiap lokasi memiliki parameter sendiri, jumlah parameternya lebih besar daripada `Conv2D`. Hal ini menjadi dasar perbandingan shared parameter dan non-shared parameter.



Gambar 4: Perbedaan parameter sharing pada `Conv2D` dan `LocallyConnected2D`.

Cuplikan ide utama `LocallyConnected2D`

```
position = i * out_w + j
patch_flat = patch.reshape(batch_size, -1)
local_kernel = kernel[position]        # (kh * kw * channels, filters)
output[:, i, j, :] = patch_flat @ local_kernel
```

Perbedaan penting dari `Conv2D` ada pada variabel `position`. Setiap lokasi output mengambil `local_kernel` yang berbeda. Dengan demikian, pola yang dipelajari pada posisi kiri atas tidak otomatis dipakai pada posisi lain. Ini membuat model lebih fleksibel, tetapi jumlah parameter bertambah secara signifikan.

Layer pooling terdiri atas `MaxPooling2D`, `AveragePooling2D`, `GlobalMaxPooling2D`, dan `GlobalAveragePooling2D`. Pooling lokal memakai jendela geser, sedangkan global pooling mereduksi seluruh dimensi spasial menjadi satu nilai per kanal. Secara umum, max pooling mempertahankan respons fitur terkuat, sedangkan average pooling merangkum respons rata-rata dalam jendela; keduanya dapat membantu peringkasan fitur dan invariansi lokal dengan karakter yang berbeda [3]. `Flatten` mengubah tensor spasial menjadi vektor satu dimensi dengan urutan *row-major*, konsisten dengan NumPy dan Keras.

Cuplikan pooling

```
window = inputs[:, h_start:h_end, w_start:w_end, :]
if mode == "max":
    output[:, i, j, :] = window.max(axis=(1, 2))
else:
    output[:, i, j, :] = window.mean(axis=(1, 2))
```

Pooling tidak memiliki parameter yang dilatih. Layer ini hanya merangkum informasi lokal pada setiap kanal. Max pooling mempertahankan aktivasi paling kuat, sedangkan average pooling mengambil rata-rata aktivasi dalam jendela. Perbedaan ini menjadi salah satu variasi eksperimen CNN.

Pada eksperimen Keras, blok CNN dibangun dari beberapa kombinasi `Conv2D` atau `LocallyConnected2D`, pooling, `GlobalAveragePooling2D`, dense hidden layer, dan classifier softmax. Variasi kedalaman dan ukuran kernel dikaitkan dengan kemampuan model membangun receptive field efektif. Arsitektur VGG menunjukkan bahwa tumpukan filter kecil dapat membentuk receptive field lebih besar sambil menjaga struktur komputasi yang sederhana [18]. Fungsi *loss* yang digunakan adalah *sparse categorical crossentropy*, sedangkan optimizer yang digunakan adalah Adam. Setelah pelatihan, bobot disimpan dalam format `.weights.h5` dan riwayat pelatihan disimpan sebagai `.history.json`. File riwayat ini menyimpan konfigurasi, training loss, validation loss, macro F1 pada test split, confusion matrix, dan nama kelas. File inilah yang kemudian dipakai oleh `summarize_results.py` untuk membuat tabel ringkasan dan plot loss.

3.7 Metrik Evaluasi

Untuk klasifikasi gambar, metrik utama adalah macro F1. Macro F1 dipilih karena setiap kelas diberi bobot yang sama ketika skor akhir dihitung. Definisi ini sejalan dengan formulasi F1 sebagai rata-rata harmonik precision dan recall per kelas [17]. Untuk setiap kelas c, precision dan recall dihitung sebagai:

$$P_c = \frac{TP_c}{TP_c + FP_c}, \qquad R_c = \frac{TP_c}{TP_c + FN_c}. \tag{2}$$

F1 per kelas kemudian dihitung dengan:

$$F1_c = \frac{2P_cR_c}{P_c + R_c}. \tag{3}$$

Macro F1 adalah rata-rata $F1_c$ untuk seluruh kelas. Untuk captioning, metrik yang dipakai adalah BLEU-4 dan METEOR. BLEU-4 menilai kecocokan n-gram sampai panjang empat sehingga sensitif terhadap kecocokan frasa yang lebih panjang [14]. METEOR lebih toleran karena mempertimbangkan kecocokan unigram, recall, dan fragmentasi urutan [1]. Karena model captioning pada eksperimen ini kecil, METEOR menjadi indikator tambahan yang lebih informatif saat BLEU-4 sangat rendah.

3.8 Forward Propagation Simple RNN dan LSTM

Simple RNN menggunakan hidden state untuk merepresentasikan konteks temporal dari token-token sebelumnya [4]. Pembaruan state ditulis sebagai:

$$\mathbf{h}_t = \tanh(\mathbf{x}_t\mathbf{W}_x + \mathbf{h}_{t-1}\mathbf{W}_h + \mathbf{b}). \tag{4}$$

Persamaan tersebut menunjukkan bahwa hidden state baru dibentuk dari input saat ini dan hidden state sebelumnya. Implementasi mendukung beberapa layer recurrent dengan opsi `return_sequences`. Bobot dimuat dari Keras dengan urutan `kernel`, `recurrent_kernel`, dan `bias`.

Cuplikan Simple RNN cell

```
def step(self, x_t, h_tm1):
    z = x_t @ self.kernel.data + h_tm1 @ self.recurrent_kernel.data
    if self.use_bias:
        z = z + self.bias.data
    return np.tanh(z)
```

Pada setiap timestep, input saat ini `x_t` dan hidden state sebelumnya `h_tm1` diproyeksikan ke ruang hidden yang sama. Hasil penjumlahan kemudian dilewatkan ke fungsi `tanh`. Hidden state baru menjadi ringkasan seluruh token yang sudah dibaca sampai timestep tersebut.

RNN sederhana dapat kesulitan menangkap dependensi jangka panjang karena gradien dapat mengecil atau membesar ketika dipropagasikan melalui banyak timestep [2]. LSTM dirancang untuk mengatasi masalah tersebut melalui cell state dan mekanisme gate [6]. LSTM memakai input gate, forget gate, kandidat cell state, dan output gate:

$$\mathbf{i}_t = \sigma(\mathbf{x}_t\mathbf{W}_{xi} + \mathbf{h}_{t-1}\mathbf{W}_{hi} + \mathbf{b}_i), \tag{5}$$

$$\mathbf{f}_t = \sigma(\mathbf{x}_t\mathbf{W}_{xf} + \mathbf{h}_{t-1}\mathbf{W}_{hf} + \mathbf{b}_f), \tag{6}$$

$$\tilde{\mathbf{c}}_t = \tanh(\mathbf{x}_t\mathbf{W}_{xc} + \mathbf{h}_{t-1}\mathbf{W}_{hc} + \mathbf{b}_c), \tag{7}$$

$$\mathbf{o}_t = \sigma(\mathbf{x}_t\mathbf{W}_{xo} + \mathbf{h}_{t-1}\mathbf{W}_{ho} + \mathbf{b}_o), \tag{8}$$

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t, \tag{9}$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t). \tag{10}$$

Persamaan cell state memperlihatkan jalur memori yang lebih langsung: forget gate mengatur memori lama, sedangkan input gate mengatur informasi baru yang masuk. Format bobot LSTM mengikuti Keras, yaitu gabungan gerbang dalam satu matriks dengan urutan `i`, `f`, `c`, dan `o`. Hidden state dan cell state awal diinisialisasi dengan nol.

Cuplikan LSTM cell

```
z = x_t @ self.kernel.data + h_tm1 @ self.recurrent_kernel.data
if self.use_bias:
    z = z + self.bias.data

i, f, c_bar, o = np.split(z, 4, axis=-1)
i = sigmoid(i)
f = sigmoid(f)
c_bar = np.tanh(c_bar)
o = sigmoid(o)
c_t = f * c_tm1 + i * c_bar
h_t = o * np.tanh(c_t)
```

LSTM memisahkan aliran informasi menjadi hidden state dan cell state. Forget gate menentukan bagian memori lama yang dipertahankan, input gate menentukan informasi baru yang masuk, dan output gate menentukan bagian cell state yang muncul sebagai hidden state. Mekanisme ini menjadi alasan LSTM lebih tahan terhadap masalah *vanishing gradient* dibandingkan Simple RNN.

3.9 Arsitektur Image Captioning

Decoder captioning menerapkan metode *pre-inject*. Pada pendekatan ini, fitur gambar diproyeksikan ke ruang embedding dan dimasukkan sebagai timestep awal sebelum token caption, sesuai keluarga arsitektur encoder-decoder untuk captioning [20, 19]. Fitur gambar $\mathbf{v}$ diproyeksikan ke ruang embedding:

$$\mathbf{x}_{-1} = \mathbf{v}\mathbf{W}_{img} + \mathbf{b}_{img}. \tag{11}$$

Token caption diproses oleh `Embedding`, kemudian hasil embedding digabungkan setelah $\mathbf{x}_{-1}$. Dengan demikian, input recurrent menjadi:

$$[\mathbf{x}_{-1}, \text{emb}(\texttt{<start>}), \text{emb}(w_1), \ldots, \text{emb}(w_{T-1})]. \tag{12}$$

Output setiap timestep dipetakan oleh dense layer dengan aktivasi softmax untuk menghasilkan distribusi probabilitas atas vocabulary. Layer embedding dan dense mengikuti bentuk operasi Keras, tetapi forward pass inferensi direplikasi dengan NumPy [9, 8]. Saat pelatihan Keras, target digeser satu posisi melalui *teacher forcing*. Saat inferensi dengan scratch implementation, decoder mendukung *greedy decoding* dan *beam search*.

Cuplikan pembentukan input decoder

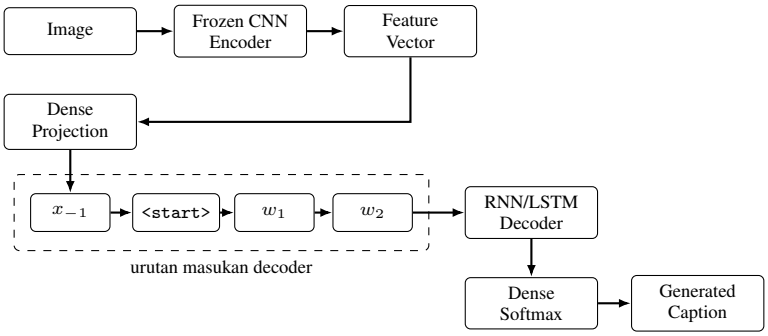
```
def build_inputs(self, features, token_ids):
    feats = self._as_feature_batch(features)
    tokens = np.asarray(token_ids, dtype=np.int64)
    image_step = self.image_projection(feats)[:, None, :]
    embedded = self.embedding(tokens)
    return np.concatenate([image_step, embedded], axis=1)
```

Fungsi ini adalah inti dari metode *pre-inject*. Fitur gambar lebih dahulu diproyeksikan ke dimensi embedding, kemudian diberi dimensi timestep dengan `[:, None, :]`. Setelah itu, embedding token caption ditempelkan setelah timestep gambar. Hasil akhirnya adalah sequence dengan panjang $T + 1$, yaitu satu timestep untuk gambar dan T timestep untuk token.

Cuplikan greedy decoding

```
sequences = [[start_id] for _ in range(batch_size)]
for _ in range(max_length):
    current = pad_sequences(sequences, value=0, padding="post")
    next_ids = self.next_token_distribution(feats, current).argmax(axis=-1)
    for index, token in enumerate(next_ids.tolist()):
        sequences[index].append(int(token))
        if token == end_id:
            finished[index] = True
    if all(finished):
        break
```

Pada decoding, model tidak menerima seluruh caption rujukan. Model mulai dari token `<start>`, memprediksi token berikutnya, kemudian token prediksi tersebut dimasukkan kembali sebagai input pada timestep selanjutnya. Proses berhenti ketika seluruh sampel menghasilkan token `<end>` atau panjang maksimum tercapai.



Gambar 5: Arsitektur captioning *pre-inject*. Fitur gambar menjadi timestep awal sebelum token caption.

Pada *greedy decoding*, model selalu memilih token dengan probabilitas terbesar pada setiap timestep. Metode ini cepat, tetapi dapat terjebak pada pilihan lokal. Pada *beam search*, beberapa kandidat urutan dipertahankan secara bersamaan sehingga decoder tidak langsung membuang kandidat yang sedikit lebih rendah pada timestep awal. Setiap kandidat menyimpan akumulasi log-probability yang

dinormalisasi terhadap panjang urutan. Pada eksperimen ini, beam width yang dipakai adalah 3. Implementasi beam search menjadi bagian bonus karena memberi alternatif decoding selain greedy.

Algoritma 1: Greedy decoding pada decoder captioning

Input: Fitur gambar $\mathbf{v}$, token awal `<start>`, token akhir `<end>`, panjang maksimum T
Output: Urutan token caption
$S \leftarrow$ [`<start>`];
for $t \leftarrow 1$ **to** T **do**
  Hitung distribusi $p(w_t \mid \mathbf{v}, S)$ dengan decoder;
  $w_t \leftarrow \arg\max_w p(w \mid \mathbf{v}, S)$;
  Tambahkan w_t ke S;
  if $w_t =$ *`<end>`* **then**
    break;
  end
end
return S *tanpa token awal*;

4 Hasil Pengujian

4.1 Pengujian Unit dan Kesetaraan Keras

Pengujian otomatis dilakukan dengan `pytest`. Seluruh 72 pengujian berhasil dijalankan. Pengujian mencakup operasi tensor, layer CNN, pooling, RNN, LSTM, embedding, loss, metrik, utilitas, pemuatan bobot Keras, dan perbandingan keluaran Keras dengan scratch implementation. Hasil ini menunjukkan bahwa komponen-komponen dasar sudah berjalan konsisten pada kasus uji yang tersedia.

4.2 Hasil CNN

Eksperimen CNN dijalankan dengan konfigurasi ringan lima epoch, ukuran gambar 64×64, batch size 8, dan dense unit 32. Untuk `Conv2D`, eksperimen terbaru memakai grid search terhadap 16 kombinasi arsitektur dari variasi kedalaman dua dan tiga layer, dua ukuran filter, dua ukuran kernel, serta dua jenis pooling. Tabel 1 menampilkan hasil variasi tersebut beserta model `LocallyConnected2D` sebagai pembanding non-shared parameter.

Tabel 1: Ringkasan hasil eksperimen CNN pada Intel Image Classification.

Layer	Depth	Filter	Kernel	Pooling	Macro F1	Val. Loss
`LocallyConnected2D`	1	8	3	Max	0.5007	1.2324
	3	8-16-24	3	Max	0.6142	0.9982
`Conv2D`	2	4-8	3	Average	0.5018	1.1512
	2	4-8	3	Max	0.5413	1.0874
	2	4-8	5	Average	0.5458	1.0848
	2	4-8	5	Max	0.5884	0.9737
	2	8-16	3	Average	0.5735	1.0127
	2	8-16	3	Max	0.5998	0.9915
	2	8-16	5	Average	0.6220	0.9706
	2	8-16	5	Max	0.6637	0.9095
	3	4-8-12	3	Average	0.5692	1.0711
	3	4-8-12	3	Max	0.6573	0.9233
	3	4-8-12	5	Average	0.6590	0.9267
	3	4-8-12	5	Max	0.6817	0.8751
	3	8-16-24	3	Average	0.6456	0.9055
	3	8-16-24	3	Max	0.7221	0.7829
	3	8-16-24	5	Average	0.7217	0.7440
	3	8-16-24	5	Max	**0.7471**	**0.7037**

Variasi kedalaman menunjukkan bahwa penambahan layer konvolusi memberi peningkatan yang cukup konsisten. Rata-rata macro F1 untuk model dua layer adalah 0.5795, sedangkan rata-rata macro F1 untuk model tiga layer naik menjadi 0.6755. Pada pasangan konfigurasi yang sebanding, model tiga layer juga selalu lebih baik daripada model dua layer. Sebagai contoh, konfigurasi small, kernel 5, max pooling meningkat dari 0.5884 menjadi 0.6817, sedangkan konfigurasi medium, kernel 5, max pooling meningkat dari 0.6637 menjadi 0.7471. Hal ini menunjukkan bahwa kedalaman tambahan membantu model menangkap pola spasial yang lebih kompleks. Interpretasi ini selaras dengan gagasan bahwa layer awal cenderung menangkap pola lokal sederhana, sedangkan layer lebih dalam dapat menggabungkannya menjadi fitur yang lebih abstrak [5, 18].

Variasi jumlah filter juga memperlihatkan tren peningkatan. Kelompok filter small, yaitu 4-8 pada dua layer dan 4-8-12 pada tiga layer, memperoleh rata-rata macro F1 0.5931. Kelompok filter medium, yaitu 8-16 pada dua layer dan 8-16-24 pada tiga layer, memperoleh rata-rata macro F1 0.6619. Pada hampir semua pasangan kedalaman, kernel, dan pooling yang sama, filter medium lebih baik daripada filter small. Dengan ukuran model yang masih kecil, kapasitas tambahan memberi ruang representasi yang lebih baik. Akan tetapi, konfigurasi ini belum cukup besar untuk menyimpulkan titik jenuh kapasitas.

Variasi ukuran kernel menunjukkan bahwa kernel 5 lebih baik daripada kernel 3 pada grid terbaru. Rata-rata macro F1 kernel 3 adalah 0.6013, sedangkan kernel 5 mencapai 0.6537. Pada seluruh pasangan konfigurasi dengan kedalaman, filter, dan pooling yang sama, kernel 5 memberi macro F1 lebih tinggi daripada kernel 3. Kemungkinan, receptive field yang lebih besar membantu model ringan melihat konteks gambar yang lebih luas. Namun, eksperimen lanjutan tetap perlu membandingkan kernel besar dengan tumpukan kernel kecil yang memiliki jumlah parameter sebanding. Tumpukan filter kecil juga dapat membentuk receptive field efektif yang besar [18].

Untuk pooling, max pooling memperoleh rata-rata macro F1 0.6502, lebih tinggi daripada average pooling yang memperoleh 0.6048. Selisih ini terlihat jelas pada beberapa konfigurasi, misalnya tiga layer small kernel 3 yang naik dari 0.5692 pada average pooling menjadi 0.6573 pada max pooling. Hasil ini menunjukkan bahwa pada konfigurasi yang diuji, mempertahankan respons fitur paling kuat lebih membantu daripada merata-ratakan seluruh respons dalam jendela pooling [3]. Walaupun demikian, average pooling masih kompetitif pada konfigurasi tiga layer medium kernel 5, dengan macro F1 0.7217 yang mendekati max pooling 0.7471.

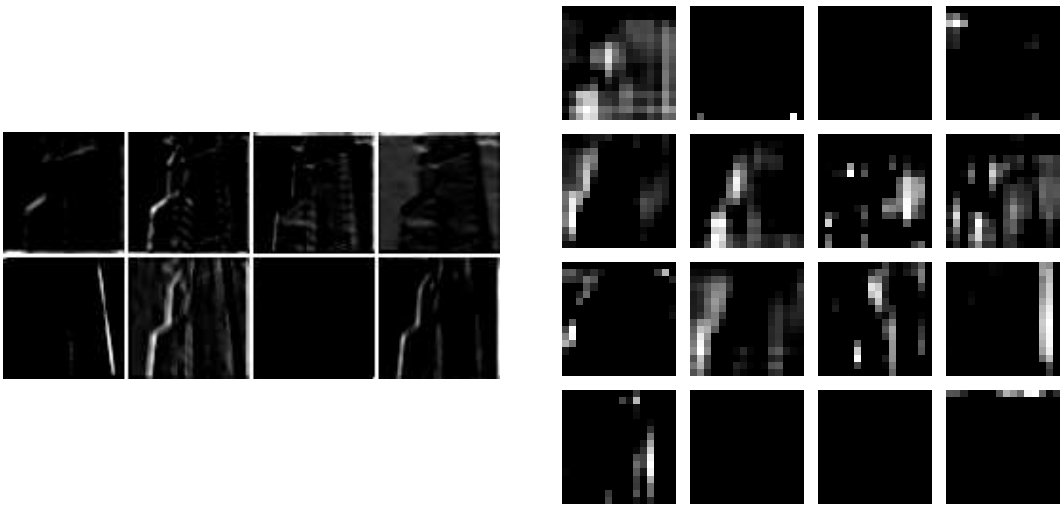
Jika dilihat dari *validation loss*, model terbaik juga memberi nilai terendah, yaitu 0.7037 pada konfigurasi tiga layer, filter 8-16-24, kernel 5, dan max pooling. Ini konsisten dengan macro F1 tertinggi yang dicapai model tersebut, yaitu 0.7471. Namun, hasil ini tetap harus dibaca dengan hati-hati karena eksperimen dilakukan dengan ukuran gambar dan jumlah epoch yang kecil. Tujuannya adalah memastikan seluruh pipeline spesifikasi berjalan dan menghasilkan artifact yang dapat dibandingkan, bukan mencari akurasi terbaik absolut. Untuk eksperimen produksi, konfigurasi yang lebih besar dan epoch lebih banyak kemungkinan akan meningkatkan performa.

4.3 Visualisasi Fitur dan Grad-CAM

Untuk melihat perilaku internal model, dilakukan visualisasi terhadap activation map pada layer-layer konvolusi dan Grad-CAM pada layer konvolusi terakhir. Analisis ini memakai model yang sama dengan eksperimen utama sehingga bobot, arsitektur, dan preprocessing gambar tetap konsisten dengan hasil pada Tabel 1. Dengan demikian, visualisasi tidak diperlakukan sebagai eksperimen terpisah, melainkan sebagai alat interpretasi terhadap model terbaik yang sudah dievaluasi secara kuantitatif.

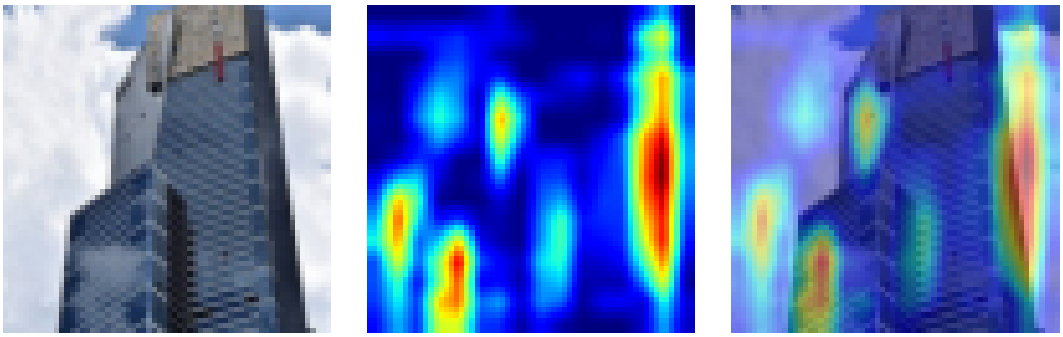
Visualisasi feature maps dilakukan dengan membangun model ekstraktor yang keluarannya adalah activation map dari layer-layer `Conv2D`. Setiap kanal dinormalisasi ke rentang 0 sampai 1, kemudian disusun menjadi grid. Gambar 6 menunjukkan contoh feature maps dari model terbaik pada sampel uji kelas *buildings*. Layer awal masih mempertahankan pola lokal yang relatif dekat dengan tepi dan tekstur sederhana, sedangkan layer lebih dalam menghasilkan aktivasi yang lebih jarang dan lebih selektif.

Grad-CAM dihitung dengan mengambil gradien skor kelas target terhadap activation map layer konvolusi terakhir. Gradien tersebut dirata-ratakan secara spasial untuk memperoleh bobot tiap kanal, kemudian bobot digunakan untuk menggabungkan activation map menjadi heatmap kelas. Pada contoh Gambar 7, model memprediksi kelas *buildings* dengan probabilitas 0.9605. Overlay Grad-



Gambar 6: Contoh feature maps dari layer konvolusi pertama dan terakhir pada model `Conv2D` terbaik.

CAM memperlihatkan area gambar yang paling berkontribusi terhadap prediksi tersebut sehingga hasil klasifikasi tidak hanya dibaca sebagai skor akhir, tetapi juga dapat ditelusuri secara visual.



Gambar 7: Input, heatmap Grad-CAM, dan overlay Grad-CAM pada sampel uji kelas *buildings*.

4.4 Perbandingan Shared dan Non-Shared Parameter

Perbandingan shared dan non-shared parameter dilakukan dengan dua sudut pandang. Pada arsitektur awal satu layer, `LocallyConnected2D` memperoleh macro F1 0.5007, sedikit lebih tinggi daripada `Conv2D` satu layer yang memperoleh 0.4937. Namun ketika arsitektur evaluasi `Conv2D` tiga layer dengan filter 8-12-16 diganti seluruh layer konvolusinya menjadi `LocallyConnected2D`, macro F1 turun dari 0.6841 menjadi 0.5943. Pada grid terbaru, `LocallyConnected2D` tiga layer dengan filter 8-16-24 juga hanya mencapai macro F1 0.6142, masih di bawah `Conv2D` sepadan yang mencapai 0.7221 pada kernel 3 dan max pooling. `LocallyConnected2D` memiliki jumlah parameter yang jauh lebih besar karena setiap posisi spasial memiliki bobot sendiri. Pada konfigurasi ringan ini, fleksibilitas tambahan tersebut tidak menghasilkan performa yang lebih baik daripada parameter sharing.

Secara konseptual, `Conv2D` mengasumsikan bahwa pola visual yang sama dapat muncul di berbagai lokasi gambar. Oleh karena itu, kernel yang sama digunakan ulang pada seluruh posisi spasial. Sebaliknya, `LocallyConnected2D` tidak menggunakan asumsi tersebut. Model ini lebih fleksibel, tetapi juga lebih mahal dan lebih mudah membutuhkan data yang lebih banyak. Pada dataset alam seperti Intel Image Classification, pola seperti tekstur hutan, garis bangunan, atau permukaan laut dapat muncul di berbagai lokasi. Oleh karena itu, asumsi parameter sharing cukup masuk akal [13, 5].

4.5 Perbandingan Keras dan Scratch Implementation pada CNN

Evaluasi *forward propagation* dengan scratch implementation dilakukan pada arsitektur evaluasi `Conv2D` dan pada variasi `LocallyConnected2D` yang mengganti seluruh layer konvolusi dengan layer non-shared. Hasilnya ditunjukkan pada Tabel 2.

Tabel 2: Perbandingan Keras dan scratch implementation pada CNN.

Model	Macro F1 Keras	Macro F1 Scratch	Kesepakatan Prediksi
`conv2d_3x8-12-16_k3_max`	0.6841	0.6841	1.0000
`locally_connected_3x8-12-16_k3_max`	0.5943	0.5943	1.0000

Macro F1 yang identik dan kesepakatan prediksi 1.0 menunjukkan bahwa pemuatan bobot dan implementasi *forward pass* CNN sudah konsisten dengan Keras untuk model yang diuji. Dengan kata lain, operasi `Conv2D`, `LocallyConnected2D`, pooling, global pooling, dense projection, aktivasi, dan softmax pada implementasi NumPy menghasilkan keputusan kelas yang sama dengan model pembanding.

4.6 Hasil Pelatihan RNN dan LSTM

Eksperimen captioning dijalankan pada fitur MobileNetV2 dengan decoder kecil, yaitu *embedding dimension* 64, hidden state 32 dan 64, panjang maksimum 12, batch size 8, dan lima epoch. Tabel 3 merangkum *validation loss* akhir.

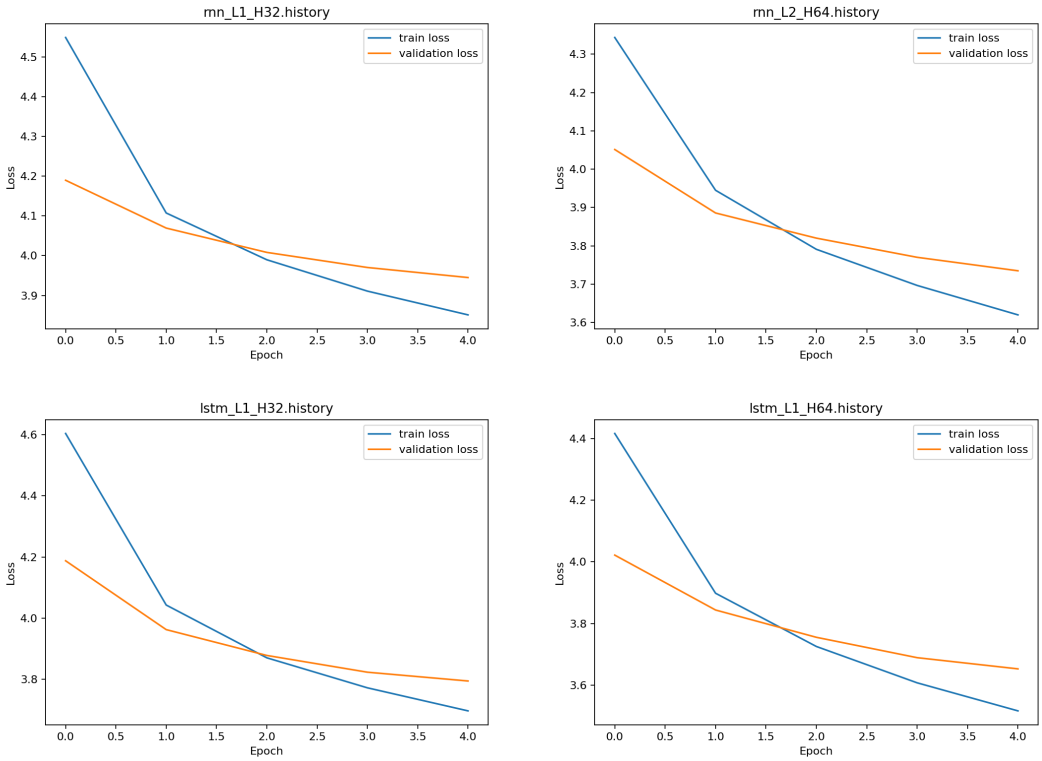
Tabel 3: Ringkasan *validation loss* decoder RNN dan LSTM.

Decoder	Layer	Hidden Size	Val. Loss
RNN	1	32	3.9449
		64	3.7142
	2	32	3.8621
		64	3.7352
	3	32	3.9467
		64	3.8042
LSTM	1	32	3.7942
		64	**3.6527**
	2	32	3.8283
		64	3.6965
	3	32	3.8763
		64	3.8075

Gambar 8 memperlihatkan bahwa penurunan loss terjadi cukup cepat pada epoch awal, tetapi jarak antara training loss dan validation loss masih terlihat pada beberapa konfigurasi. Pola ini wajar pada eksperimen kecil karena model hanya dilatih lima epoch dan vocabulary tetap cukup beragam. Kurva tersebut juga membantu membaca Tabel 3: konfigurasi dengan hidden size 64 cenderung mencapai loss lebih rendah, tetapi penambahan layer tidak otomatis membuat kurva validasi lebih baik.

Pada decoder RNN, pengaruh jumlah layer tidak menunjukkan tren peningkatan monoton. Untuk hidden size 32, dua layer memberi *validation loss* terendah sebesar 3.8621, lebih baik daripada satu layer 3.9449 dan tiga layer 3.9467. Namun, untuk hidden size 64, konfigurasi satu layer justru paling baik dengan *validation loss* 3.7142. Penambahan layer menjadi dua dan tiga layer menaikkan *validation loss* menjadi 3.7352 dan 3.8042. Hasil ini menunjukkan bahwa penambahan kedalaman pada Simple RNN belum tentu membantu pada konfigurasi pelatihan kecil.

Pada decoder LSTM, konfigurasi satu layer konsisten menjadi yang terbaik untuk kedua ukuran hidden state. LSTM satu layer dengan hidden size 32 memperoleh *validation loss* 3.7942, sedangkan dua dan tiga layer masing-masing memperoleh 3.8283 dan 3.8763. Pola yang sama muncul pada hidden size 64: satu layer memperoleh 3.6527, dua layer 3.6965, dan tiga layer 3.8075. Dengan demikian, pada eksperimen ini, mekanisme gate LSTM sudah memberi manfaat tanpa perlu menambah kedalaman recurrent yang lebih sulit dilatih.



Gambar 8: Kurva training loss dan validation loss untuk beberapa konfigurasi decoder RNN dan LSTM.

Pengaruh ukuran hidden state terlihat lebih konsisten. Pada RNN, hidden size 64 selalu menghasilkan *validation loss* lebih rendah daripada hidden size 32 untuk setiap jumlah layer: 3.7142 dibandingkan 3.9449 pada satu layer, 3.7352 dibandingkan 3.8621 pada dua layer, dan 3.8042 dibandingkan 3.9467 pada tiga layer. Hidden state yang lebih besar menyediakan ruang representasi lebih luas untuk menyimpan informasi visual dan linguistik, terutama ketika caption dibangkitkan secara autoregresif.

Pada LSTM, hidden size 64 juga selalu lebih baik daripada hidden size 32. Selisihnya terlihat pada satu layer, yaitu 3.6527 dibandingkan 3.7942, pada dua layer 3.6965 dibandingkan 3.8283, dan pada tiga layer 3.8075 dibandingkan 3.8763. Konfigurasi terbaik secara keseluruhan adalah LSTM satu layer dengan hidden size 64. Hal ini wajar karena LSTM memiliki jalur cell state yang membantu mempertahankan informasi lebih lama daripada Simple RNN [6, 2].

Perbandingan RNN dan LSTM menunjukkan bahwa LSTM umumnya lebih stabil. Pada pasangan konfigurasi yang sama, LSTM memperoleh *validation loss* lebih rendah hampir di seluruh variasi, kecuali pada tiga layer hidden size 64 dengan selisih sangat kecil. Temuan ini sesuai dengan teori bahwa deep recurrent networks dapat memperkaya representasi temporal, tetapi biasanya membutuhkan pelatihan lebih stabil, data yang memadai, dan jumlah epoch yang lebih besar agar manfaat kedalaman dapat muncul [15]. Pada captioning, kemampuan mempertahankan konteks penting karena model harus mengingat objek visual dan kata-kata yang sudah dibangkitkan sebelumnya.

Evaluasi inferensi *scratch* untuk seluruh variasi ditunjukkan pada Tabel 4. Seluruh variasi memakai greedy decoding dengan panjang maksimum 20. Nilai BLEU-4 ditulis dalam skala 10^{-5} agar perbedaan kecil antarvariasi tetap mudah dibaca.

Hasil evaluasi pada Tabel 4 menunjukkan bahwa *validation loss* tidak selalu berbanding lurus dengan BLEU-4 atau METEOR. LSTM satu layer hidden size 64 memiliki *validation loss* terbaik, tetapi BLEU-4 tertinggi diperoleh LSTM satu layer hidden size 32. Pada RNN, BLEU-4 tertinggi diperoleh konfigurasi dua layer hidden size 64, sedangkan METEOR tertinggi diperoleh RNN tiga layer hidden size 32. Pola ini menunjukkan bahwa pada model kecil, perubahan arsitektur dapat mengubah bentuk keluaran caption tanpa selalu memperbaiki kecocokan n-gram. METEOR menjadi metrik yang lebih

Tabel 4: Evaluasi *scratch* seluruh variasi decoder captioning.

Decoder	Layer	Hidden Size	BLEU-4 ($\times 10^{-5}$)	METEOR	s/gambar
RNN	1	32	0.5734	0.1389	**0.0008**
	1	64	1.0188	0.1588	0.0009
	2	32	0.4937	0.1612	0.0031
	2	64	1.1131	0.1614	0.0021
	3	32	0.0018	**0.1780**	0.0042
	3	64	0.0033	0.1423	0.0020
LSTM	1	32	**1.1667**	0.1654	0.0014
	1	64	0.6580	0.1558	0.0016
	2	32	0.8046	0.1351	0.0017
	2	64	0.5973	0.1556	0.0025
	3	32	0.0038	0.1628	0.0084
	3	64	0.0048	0.1574	0.0019

stabil untuk membaca kecenderungan kualitas karena masih memberi nilai pada kecocokan unigram, sedangkan BLEU-4 sangat mudah turun ketika frasa empat token tidak cocok dengan referensi.

Dari sisi waktu eksekusi, konfigurasi satu layer cenderung lebih cepat karena jumlah operasi recurrent lebih sedikit. RNN satu layer hidden size 32 menjadi konfigurasi tercepat dengan waktu sekitar 0.0008 detik per gambar. Penambahan layer umumnya menaikkan biaya inferensi, walaupun nilainya tidak sepenuhnya monoton karena pengukuran dilakukan pada pipeline Python dan dipengaruhi overhead pemanggilan fungsi serta panjang caption efektif yang dihasilkan.

4.7 Evaluasi Captioning: RNN vs LSTM dan Keras vs Scratch

Tabel 5 menunjukkan perbandingan terfokus antara Keras dan scratch implementation pada model satu layer hidden size 32. Nilai BLEU-4 sangat kecil karena model dilatih dengan konfigurasi ringan dan jumlah epoch terbatas, tetapi nilai METEOR masih dapat dipakai untuk membandingkan kecenderungan keluaran.

Tabel 5: Evaluasi captioning pada model RNN dan LSTM satu layer hidden size 32.

Decoder	Mode	Decoding	BLEU-4	METEOR
RNN	Scratch	Greedy	0.0000057	0.1389
	Keras		0.0000057	0.1389
LSTM	Scratch	Greedy	0.0000117	0.1654
	Keras	Greedy	0.0000117	0.1643
	Scratch	Beam	0.0000076	0.1662

LSTM memberikan METEOR yang lebih baik daripada RNN pada konfigurasi yang dibandingkan. Hal ini sesuai dengan sifat LSTM yang memiliki cell state dan gate sehingga lebih mampu mempertahankan informasi jangka panjang. RNN sederhana lebih mudah mengalami peluruhan informasi karena seluruh konteks harus dipadatkan dalam hidden state tanpa mekanisme gate eksplisit [4, 6].

Perbandingan Keras dan *scratch* pada decoding greedy menunjukkan skor yang hampir sama. Pada RNN, skor Keras dan *scratch* identik. Pada LSTM, skor METEOR berbeda tipis, yaitu 0.1654 untuk *scratch* dan 0.1643 untuk Keras. Kedekatan skor ini menunjukkan bahwa embedding lookup, recurrent update, dense projection, dan softmax output sudah berjalan selaras. Perbedaan kecil dapat berasal dari detail numerik, urutan operasi, atau perbedaan pembulatan saat inferensi.

Waktu eksekusi per gambar pada *scratch* jauh lebih rendah daripada Keras pada artifact yang dihasilkan. Hal ini tidak berarti implementasi NumPy selalu lebih cepat pada semua kondisi. Pada eksperimen ini, inferensi *scratch* menjalankan satu decoder kecil tanpa overhead graph Keras, sedangkan pemanggilan `predict` Keras berulang untuk setiap timestep memiliki overhead yang lebih besar. Jika inference Keras dibuat dalam batch besar atau memakai graph execution yang lebih optimal, perbandingan waktunya dapat berubah.

4.8 Pengaruh Panjang Maksimum Caption

Variasi panjang maksimum caption dilakukan pada LSTM *scratch* dengan greedy decoding. Hasilnya ditunjukkan pada Tabel 6.

Tabel 6: Pengaruh panjang maksimum caption terhadap skor LSTM *scratch*.

Panjang Maksimum	BLEU-4	METEOR
12	0.0000117	0.1654
20	0.0000117	0.1654
28	0.0000117	0.1654

Pada eksperimen ini, perubahan panjang maksimum tidak mengubah skor. Artinya, model cenderung menghasilkan token akhir atau pola keluaran yang sama sebelum batas panjang tercapai. Karena BLEU-4 bergantung pada kecocokan n-gram, perubahan batas panjang tidak akan memengaruhi skor jika urutan efektif yang dihasilkan tetap sama [14]. Untuk model yang lebih matang, variasi panjang maksimum biasanya lebih berpengaruh karena decoder dapat menghasilkan caption yang lebih panjang dan lebih beragam.

4.9 Analisis Kualitatif

Secara kualitatif, caption yang dihasilkan masih sederhana dan sering berulang. Gambar 9 menampilkan 10 gambar evaluasi yang digunakan pada Tabel 7. Contoh ini memakai model RNN dan LSTM satu layer hidden size 32 dengan greedy decoding agar perbedaan pola keluaran dapat dibaca langsung.

Gambar 9: Sepuluh gambar sampel yang dipakai untuk analisis kualitatif captioning.

Tabel 7: Contoh kualitatif caption RNN dan LSTM pada 10 gambar evaluasi.

No.	RNN	LSTM	Salah satu caption rujukan
1	man a in and are in of	man a and are on in of	a child in a pink dress is climbing up a set of stairs in an entry way
2	man a in and are in of	dogs a and dog in snow	a black dog and a spotted dog are fighting
3	man a in and are in of	man a and dog a in water a	a little girl covered in paint sits in front of a painted rainbow with her hands in a bowl
4	man a in and are in of	man a on is on bike a	a man lays on a bench while his dog sits by him
5	man a in and are in of	man a on is on bike a	a man in an orange hat starring at something
6	man a in and are in of	man a on is on bike a	a child playing on a rope net
7	man a in and are in of	dog a dog a dog the	a black and white dog is running in a grassy garden surrounded by a white fence
8	man a in and are in of	dog a dog a dog the	a dog shakes its head near the shore a red ball next to it
9	man a in and are in of	man a on is on bike a	a boy smiles in front of a stony wall in a city
10	man a in and are in of	dog a dog a dog the	a black dog leaps over a log

Tabel 7 memperlihatkan bahwa RNN cenderung jatuh pada pola keluaran yang hampir sama untuk banyak gambar, yaitu frasa pendek yang mengandung token umum seperti *man*, *a*, dan *in*. LSTM menghasilkan keluaran yang sedikit lebih beragam. Pada beberapa contoh, LSTM mulai menangkap objek dominan seperti *dogs* atau *dog*, tetapi struktur kalimatnya masih belum gramatikal dan masih sering berulang. Keterbatasan ini terutama disebabkan oleh konfigurasi eksperimen yang sengaja dibuat kecil: lima epoch, decoder kecil, dan vocabulary yang dibangun dari eksperimen cepat. Meskipun demikian, pipeline end-to-end sudah berjalan: gambar diproses oleh encoder, fitur dimasukkan ke decoder, token dibangkitkan secara autoregresif, dan hasilnya dapat dibandingkan dengan caption rujukan.

Perbedaan utama antara RNN dan LSTM terlihat pada stabilitas memori. LSTM cenderung memberi skor METEOR lebih tinggi pada konfigurasi pembanding karena cell state membantu mempertahankan konteks visual dan kata-kata sebelumnya. RNN tidak memiliki jalur memori eksplisit sehingga lebih rentan kehilangan informasi, terutama ketika caption lebih panjang. Namun, hasil pada Tabel 4 juga menunjukkan bahwa keunggulan ini belum sepenuhnya stabil pada semua variasi kecil. Dengan jumlah epoch dan data efektif yang terbatas, mekanisme LSTM membantu, tetapi belum cukup untuk menghasilkan caption yang benar-benar deskriptif.

4.10 Keterbatasan Eksperimen

Eksperimen pada laporan ini memakai parameter kecil agar seluruh variasi yang disyaratkan tetap dapat dijalankan dalam waktu wajar. CNN dilatih pada gambar 64×64 dengan jumlah filter kecil. Captioning memakai MobileNetV2, hidden size 32 dan 64, serta lima epoch. Konfigurasi ini cukup untuk memvalidasi implementasi, pipeline, dan pemindahan bobot, tetapi belum cukup untuk menghasilkan performa terbaik. Dengan komputasi lebih besar, beberapa peningkatan yang disarankan adalah memakai encoder visual yang lebih kuat, memperbesar hidden state, menambah epoch, melakukan regularisasi dan *early stopping*, serta mengevaluasi caption pada split train-validation-test resmi Flickr8k.

5 Kesimpulan dan Saran

Scratch implementation untuk *forward propagation* CNN, Simple RNN, dan LSTM berhasil dibuat dan diuji. Tujuan utama proyek ini adalah memvalidasi forward propagation pada scratch implementation, bukan mencapai performa state-of-the-art. Pada CNN, pemindahan bobot dari Keras ke scratch implementation NumPy menghasilkan keluaran yang konsisten, dibuktikan dengan macro F1 identik dan kesepakatan prediksi 1.0 pada model evaluasi shared dan non-shared. Eksperimen CNN menunjukkan bahwa penambahan kedalaman, filter, dan ukuran kernel dapat meningkatkan performa pada konfigurasi kecil. Parameter sharing pada `Conv2D` juga terbukti lebih efisien daripada `LocallyConnected2D` pada arsitektur pembanding yang diuji.

Pada captioning, pipeline *pre-inject* berhasil dijalankan untuk RNN dan LSTM. LSTM memberikan hasil lebih baik daripada RNN pada skor METEOR, sesuai dengan keunggulan mekanisme gate dan cell state. Namun, skor BLEU-4 masih sangat rendah karena eksperimen memakai konfigurasi ringan. Untuk peningkatan berikutnya, pelatihan dapat dilakukan lebih lama, encoder visual dapat dibuat lebih kuat, ukuran hidden state dapat diperbesar, split data dapat dibuat lebih ketat, regularisasi dan *early stopping* dapat ditambahkan, serta evaluasi kualitatif dapat diperluas dengan contoh gambar yang lebih representatif.

6 Pembagian Tugas

Tabel 8: Pembagian tugas anggota kelompok.

NIM	Nama	Kontribusi
13523097	Shanice Feodora Tjahjono	Implementasi layer CNN, pengujian pooling, dan penyusunan analisis CNN.
13523105	Muhammad Fathur Rizky	Implementasi RNN/LSTM, pipeline captioning, dan evaluasi Keras vs *scratch*.
13523121	Ahmad Wicaksono	Penyusunan eksperimen, metrik, notebook, ringkasan hasil, dan laporan.

Referensi

[1] Satanjeev Banerjee and Alon Lavie. Meteor: An automatic metric for mt evaluation with improved correlation with human judgments. In *Proceedings of the ACL Workshop on Intrinsic and Extrinsic Evaluation Measures for Machine Translation and/or Summarization*, pages 65–72, 2005.

[2] Yoshua Bengio, Patrice Simard, and Paolo Frasconi. Learning long-term dependencies with gradient descent is difficult. *IEEE Transactions on Neural Networks*, 5(2):157–166, 1994. doi: 10.1109/72.279181.

[3] Y-Lan Boureau, Jean Ponce, and Yann LeCun. A theoretical analysis of feature pooling in visual recognition. In *Proceedings of the 27th International Conference on Machine Learning*, pages 111–118. Omnipress, 2010. URL `https://icml.cc/Conferences/2010/papers/638.pdf`.

[4] Jeffrey L. Elman. Finding structure in time. *Cognitive Science*, 14(2):179–211, 1990. doi: 10.1207/s15516709cog1402_1.

[5] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. URL `https://www.deeplearningbook.org`.

[6] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9 (8):1735–1780, 1997. doi: 10.1162/neco.1997.9.8.1735.

[7] Keras Team. Conv2d layer. `https://keras.io/2/api/layers/convolution_layers/convolution2d/`, 2026. Accessed: 2026-05-03.

[8] Keras Team. Dense layer. `https://keras.io/api/layers/core_layers/dense/`, 2026. Accessed: 2026-05-03.

[9] Keras Team. Embedding layer. `https://keras.io/api/layers/core_layers/embedding/`, 2026. Accessed: 2026-05-03.

[10] Keras Team. Lstm layer. `https://keras.io/api/layers/recurrent_layers/lstm/`, 2026. Accessed: 2026-05-03.

[11] Keras Team. The base layer class. `https://keras.io/api/layers/base_layer/`, 2026. Accessed: 2026-05-03.

[12] Keras Team. Simplernn layer. `https://keras.io/api/layers/recurrent_layers/simple_rnn/`, 2026. Accessed: 2026-05-03.

[13] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998. doi: 10.1109/5.726791.

[14] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. Bleu: A method for automatic evaluation of machine translation. In *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*, pages 311–318. Association for Computational Linguistics, 2002. doi: 10.3115/1073083.1073135.

[15] Razvan Pascanu, Caglar Gulcehre, Kyunghyun Cho, and Yoshua Bengio. How to construct deep recurrent neural networks. In *Proceedings of the Second International Conference on Learning Representations*, 2014. URL https://arxiv.org/abs/1312.6026.

[16] Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov, and Liang-Chieh Chen. Mobilenetv2: Inverted residuals and linear bottlenecks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 4510–4520, 2018.

[17] scikit-learn developers. sklearn.metrics.f1_score. https://scikit-learn.org/stable/modules/generated/sklearn.metrics.f1_score.html, 2026. Accessed: 2026-05-03.

[18] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. In *International Conference on Learning Representations*, 2015. URL https://arxiv.org/abs/1409.1556.

[19] Marc Tanti, Albert Gatt, and Kenneth P. Camilleri. Where to put the image in an image caption generator. *Natural Language Engineering*, 24(3):467–489, 2018. doi: 10.1017/S1351324918000098.

[20] Oriol Vinyals, Alexander Toshev, Samy Bengio, and Dumitru Erhan. Show and tell: A neural image caption generator. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 3156–3164, 2015. doi: 10.1109/CVPR.2015.7298935.

FORMAT DEKLARASI PENGGUNAAN AI
SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, saya/kami*:

Nama/NIM : Shanice Feodora Tjahjono/13523097
Fakultas/Sekolah : STEI-K
Kode Mata Kuliah : IF3270
Nama Mata Kuliah : Pembelajaran Mesin
Judul Tugas/Proposal : IF3270 Tugas Besar 2 Pembelajaran Mesin

Menyatakan bahwa saya menggunakan AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

Jika Ya, maka alat AI/kecerdasan buatan yang digunakan adalah:

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	Tidak	-
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Ya	2 – AI-assisted idea generation and structuring
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	Ya	2 – AI-assisted idea generation and structuring
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	Tidak	-
Lainnya, Jelaskan:		

Saya/Kami* juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung, 5 Mei 2026

Shanice Feodora Tjahjono

13523097

FORMAT DEKLARASI PENGGUNAAN AI
SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, saya/kami*:

Nama/NIM : Muhammad Fathur Rizky/13523105
Fakultas/Sekolah : STEI-K
Kode Mata Kuliah : IF3270
Nama Mata Kuliah : Pembelajaran Mesin
Judul Tugas/Proposal : IF3270 Tugas Besar 2 Pembelajaran Mesin

Menyatakan bahwa saya menggunakan AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

Jika Ya, maka alat AI/kecerdasan buatan yang digunakan adalah:

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	Tidak	-
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Ya	2 – AI-assisted idea generation and structuring
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	Ya	2 – AI-assisted idea generation and structuring
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	Tidak	-
Lainnya, Jelaskan:		

Saya/Kami* juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung, 5 Mei 2026

Muhammad Fathur Rizky
13523105

FORMAT DEKLARASI PENGGUNAAN AI
SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, saya/kami*:

Nama/NIM : Ahmad Wicaksono/13523121
Fakultas/Sekolah : STEI-K
Kode Mata Kuliah : IF3270
Nama Mata Kuliah : Pembelajaran Mesin
Judul Tugas/Proposal : IF3270 Tugas Besar 2 Pembelajaran Mesin

Menyatakan bahwa saya menggunakan AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

Jika Ya, maka alat AI/kecerdasan buatan yang digunakan adalah:

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	Tidak	-
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Ya	2 – AI-assisted idea generation and structuring
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	Ya	2 – AI-assisted idea generation and structuring
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	Tidak	-
Lainnya, Jelaskan:		

Saya/Kami* juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung, 5 Mei 2026

Ahmad Wicaksono

13523121